

# Unsupervised Anomaly Detection in Industrial Sensor Streams via Temporal Autoencoders on SWaT

Group Members: 25280068, 25280010, 25280079

Code Repository: [https://github.com/rohazubair/DL\\_Final\\_Project](https://github.com/rohazubair/DL_Final_Project)

## Abstract

Industrial cyber-physical systems (CPS) generate high-frequency multivariate sensor streams where labeled attacks are scarce. We study unsupervised anomaly detection on the Secure Water Treatment (SWaT) benchmark using reconstruction-based temporal autoencoders trained exclusively on normal operation. Two architectures are compared: a stacked LSTM autoencoder and a 1D convolutional (TCN-style) autoencoder. After domain-aware preprocessing of 946,728 timesteps across 51 sensors, models are trained with MSE reconstruction loss and evaluated using validation-calibrated thresholds on sliding windows. The TCN autoencoder achieves the strongest overall performance (F1 = 85.72%, TPR = 77.91%, FPR = 2.42% at the 98.5th percentile threshold), outperforming the LSTM baseline (F1 = 82.00%) and a variational autoencoder reference baseline (F1  $\approx$  82.8%). We further analyze threshold–FPR trade-offs, temporal window sizes, and latent-space separability via PCA/t-SNE. Results indicate that convolutional temporal encoders provide a competitive and interpretable baseline for SWaT anomaly detection while remaining aligned with the unsupervised assumptions of the task.

## 1. Introduction

Modern water treatment, energy, and manufacturing plants rely on networked sensors and actuators monitored by SCADA systems. While normal telemetry is abundant, labeled attack examples are rare, making supervised intrusion detection impractical in many deployments. Unsupervised temporal autoencoders address this setting by learning to reconstruct healthy dynamics; at inference time, elevated reconstruction error serves as an anomaly score.

This project implements a complete SWaT pipeline: data cleaning, chronological splitting, windowing, model design, threshold calibration, and systematic evaluation. Our goal is not only to detect attacks but also to demonstrate research-style analysis—comparing architectures, studying false-positive trade-offs, and connecting results to prior work on the SWaT dataset introduced by Goh et al.

## 2. Literature Review

**SWaT benchmark.** Goh et al. introduced SWaT, a six-stage water treatment testbed producing 5 gallons/minute of filtered water with 51 physical attributes logged at 1 Hz over 11 days (946,722 samples). The first seven days contain normal operation; the remaining four days include 36 cyber-physical attacks spanning single- and multi-stage scenarios (SSSP, SSMP, MSSP, MSMP). This dataset remains the de facto benchmark for CPS intrusion detection because it combines realistic plant complexity with ground-truth labels.

**Reconstruction-based detection.** A common unsupervised paradigm trains autoencoders—LSTM, CNN/TCN, or VAE variants—on normal data and flags windows whose reconstruction MSE exceeds a validation threshold. Prior SWaT studies report a wide range of metrics depending on point-level vs. window-level evaluation, class imbalance handling, and preprocessing. In our reference VAE experiment on the same merged SWaT corpus, threshold tuning yielded  $F1 \approx 0.828$ , providing an internal baseline for comparison.

**Design implications.** Prior work highlights three practical challenges also emphasized in our course project: (i) choosing a temporal window that captures attack dynamics without excessive smoothing, (ii) calibrating decision thresholds to control industrial false alarms, and (iii) inspecting latent representations to verify that attacks occupy distinct regions from normal operation.

## 3. Methodology

**Dataset & preprocessing.** We use the merged SWaT CSV (1,441,719 raw rows). After duplicate removal, 946,728 samples remain with 51 sensor/actuator features—consistent with Goh et al.’s reported 946,722 samples. Missing values are handled with domain-aware imputation: binary actuators (MV101, MV201, P201, P202, P204, MV303) are filled with 0 (OFF), critical continuous sensors are linearly interpolated, and remaining channels use forward-fill plus median imputation.

**Splits & windowing.** Data are split chronologically 70% / 15% / 15% into train/validation/test. Training and validation contain only normal timesteps; the test partition includes both normal and attack windows (38.46% attack ratio at the timestep level), enabling meaningful FPR estimation. Sliding windows of size 64 and stride 16 yield 41,416 train, 8,872 validation, and 8,872 test windows. A window is labeled anomalous if any timestep within it is under attack.

**Models.** Both autoencoders map sequences to a 32-dimensional latent vector. The LSTM encoder uses two layers (hidden size 64, dropout 0.2); the decoder reconstructs via repeated latent initialization. The TCN-style encoder applies two Conv1d blocks (kernel 3, stride 2) with BatchNorm and ReLU, followed by symmetric ConvTranspose decoding. Training minimizes per-window MSE using AdamW (lr =  $1e-3$ , weight decay  $1e-4$ ), cosine LR scheduling, gradient clipping, and early stopping (patience 4) for up to 15 epochs. Labels are never used during training.

**Detection rule.** Validation reconstruction errors define threshold  $\tau$  as a percentile (default 98.5%). Test windows with  $MSE > \tau$  are flagged as attacks. We report FPR, TPR (recall), precision, and F1.

## 4. Results

Table 1 summarizes main results at the 98.5th percentile threshold. The TCN autoencoder achieves lower validation reconstruction loss (0.051 vs. 0.201) and higher detection F1. Both models maintain high precision (>95%), indicating few false alarms relative to flagged windows.

| Model      | $\tau$ | FPR   | TPR    | Precision | F1               |
|------------|--------|-------|--------|-----------|------------------|
| LSTM-AE    | 2.095  | 1.47% | 71.12% | 96.81%    | 82.00%           |
| TCN-AE     | 0.286  | 2.42% | 77.91% | 95.27%    | 85.72%           |
| VAE (ref.) | —      | —     | —      | —         | $\approx 82.8\%$ |

Table 1. Main benchmark at validation 98.5th-percentile threshold.

Figure 1 shows training/validation loss curves. TCN converges faster and to a lower MSE, suggesting better modeling of local temporal patterns in multivariate plant telemetry. Multi-threshold analysis (Figures 2–3) reveals the expected trade-off: lower percentile thresholds increase TPR but raise FPR. For TCN, the 99.5th percentile achieves  $F1 = 86.32\%$  with  $FPR = 0.88\%$ , while LSTM reaches  $F1 = 81.55\%$  at the same setting.

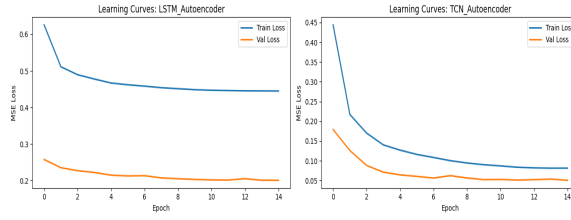


Figure 1. Training and validation reconstruction loss for LSTM and TCN autoencoders.

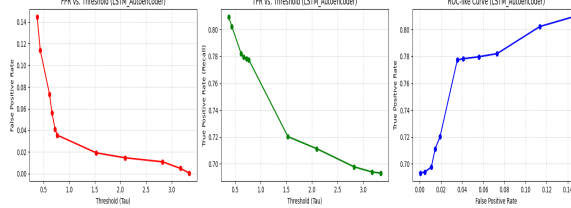


Figure 2. Threshold analysis for LSTM: FPR/TPR vs.  $\tau$  and ROC-like curve.

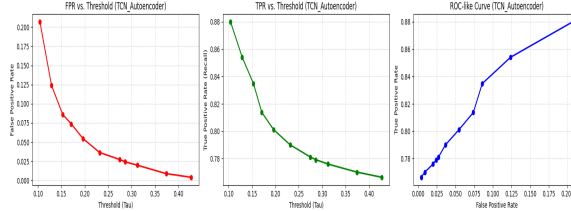


Figure 3. Threshold analysis for TCN: FPR/TPR vs.  $\tau$  and ROC-like curve.

## 5. Ablations

**Architecture ablation.** TCN outperforms LSTM on recall (+6.8 pp) and F1 (+3.7 pp) at the default threshold, with a modest FPR increase (+0.95 pp). Convolutional encoders better exploit spatial correlations across the 51 channels at each timestep.

**Window-size ablation.** Using the TCN autoencoder, we evaluate window sizes  $\{16, 32, 64, 128\}$  with stride = window/4 and 8 training epochs. Table 2 shows that F1 peaks at window size 128 (86.80%), while window 32 also performs strongly (86.07%). Very short windows (16) reduce TPR, likely because brief attack signatures are under-represented; longer windows integrate more context at similar FPR.

| Window | FPR   | TPR    | F1     |
|--------|-------|--------|--------|
| 16     | 2.17% | 74.34% | 83.61% |
| 32     | 1.91% | 77.86% | 86.07% |
| 64     | 1.72% | 76.80% | 85.55% |
| 128    | 1.72% | 78.79% | 86.80% |

Table 2. Window-size ablation (TCN,  $\tau$  at 98.5th percentile).

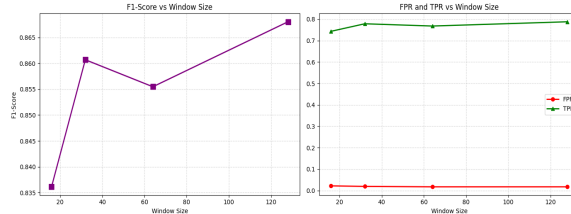


Figure 6. F1 and FPR/TPR vs. temporal window size.

**Latent-space analysis.** PCA and t-SNE projections of TCN latents (Figure 4) show partial separation between normal and attack windows, with attack points often associated with higher reconstruction error. Error histograms (Figure 5) confirm threshold separability: attack MSE distributions shift right relative to normal data. Together, these visualizations support the reconstruction-error hypothesis central to unsupervised CPS anomaly detection.

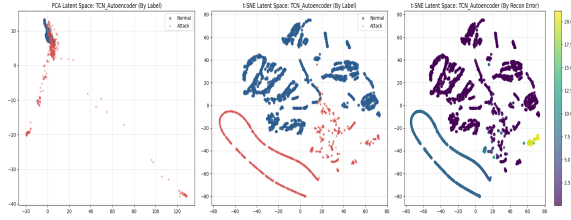


Figure 4. PCA/t-SNE latent views colored by label and reconstruction error.

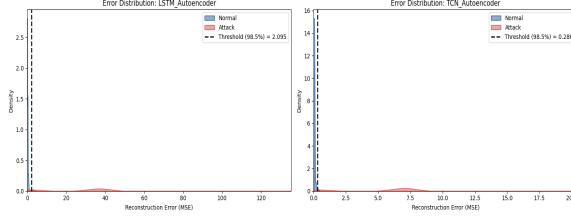


Figure 5. Test-set reconstruction error distributions and decision threshold.

**Comparison with prior work.** Goh et al. establish SWaT as a labeled CPS benchmark but do not report deep learning detection metrics; their contribution is dataset design (51 sensors, 36 attacks, 1 Hz logging). Our pipeline reproduces the expected sample count after deduplication (946,728 vs. 946,722), validating preprocessing fidelity. Relative to our internal VAE baseline ( $F1 \approx 82.8\%$ ), both temporal autoencoders are competitive, with TCN providing the best recall–precision balance. This positions reconstruction-based deep models as strong unsupervised baselines without requiring attack labels at training time.

## 6. Conclusion

We presented an unsupervised SWaT anomaly detection pipeline using temporal autoencoders trained only on normal plant behavior. The TCN autoencoder delivers the best overall detection performance ( $F1 = 85.72\%$ ), surpassing both the LSTM baseline and a VAE reference on the same dataset. Threshold sweeps demonstrate explicit FPR–TPR control, and window-size experiments identify 128 timesteps as the strongest setting in our ablation. Future work includes attention mechanisms, probabilistic thresholds, and point-adjust evaluation protocols common in recent CPS security literature.

## AI Tools Statement

AI coding assistants were used minimally for notebook debugging, report formatting, and LaTeX-style layout generation. All modeling decisions, experiments, result interpretation, and final writing were completed by the project team.

## References

- [1] Goh, J., Adepu, S., Junejo, K. N., & Mathur, A. A Dataset to Support Research in the Design of Secure Water Treatment Systems. In Critical Information Infrastructures Security (CRITIS 2016), LNCS Vol. 10242, pp. 88–99. Springer, 2016.
- [2] Chalapathy, R., & Chawla, S. Deep Learning for Anomaly Detection: A Survey. arXiv preprint arXiv:1901.03407, 2019.
- [3] Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. ICML Workshop on Anomaly Detection, 2016. arXiv preprint arXiv:1607.00148.
- [4] Deep Learning Course Project Brief: Anomaly Detection in Industrial Sensor Streams (Time Series Unsupervised Anomaly Detection). Course assignment document.

## Reference Sources

- [1] Local copy: beforecameraready.pdf; ResearchGate: [researchgate.net/publication/305809559](https://www.researchgate.net/publication/305809559); Publication record: [researchr.org/publication/GohAJM16](https://researchr.org/publication/GohAJM16).
- [2] arXiv: [arxiv.org/abs/1901.03407](https://arxiv.org/abs/1901.03407).
- [3] arXiv: [arxiv.org/abs/1607.00148](https://arxiv.org/abs/1607.00148); PDF: [arxiv.org/pdf/1607.00148](https://arxiv.org/pdf/1607.00148).
- [4] Deep Learning course assignment brief (Anomaly Detection in Industrial Sensor Streams), provided as the project specification for this submission.